

Storybook as your Component Contract

Drift

The Problem

Design $\longleftrightarrow$ Code drift

Code $\longleftrightarrow$ Usage

Docs $\longleftrightarrow$ Reality

Drift is inevitable...unless something enforces the truth

Anything that isn't tied to the code...will drift

Fundamentals (Done right)

✗ Imperative / Not testable

```
const handleClick = () => {  
  clicked = true;  
  console.log('clicked',  
    clicked);  
};  
  
return <Button label="Click  
Me" onClick={handleClick} />
```

```
export const Default = {  
  args: {  
    label: 'Click Me',  
    isDisabled: false,  
  },  
};
```

Controls == Exploration

Exploration == Understanding

Autodocs

If you're writing docs manually, you're already losing

Autodocs == No Drift

Prop tables from Typescript
Live Examples
Always up to date

Storybook

Find components / +

Configure your project

EXAMPLE

Button

Docs

Primary

Secondary

Large

Small

Header

Docs

Logged In

Logged Out

Page

Logged In

Logged Out

localhost

Button

Primary UI component for user interaction

Button

Show code

Name	Description	Default	Control
onClick	Optional click handler <code>() => void</code>	-	-
primary	Is this the principal call to action on the page? <code>boolean</code>	<code>false</code>	FalseTrue
label*	Button contents <code>string</code>	-	Button
backgroundColor	What background color to use <code>string</code>	-	#FB8A62
size	How large should the button be? <code>union</code>	<code>'medium'</code>	small medium large

STORIES

Primary

Button

Show code

Play Functions

```
const OpensModal = {  
  args: { label: 'Open' },  
  play: async({ele}) => {  
    const canvas = within(ele);  
    await userEvent.click(canvas  
      .getByText('Open'))  
    expect(canvas.getByRole('dialog'))  
      .toBeVisible();  
  }  
}
```

Visual Regression Testing

Regression Testing For

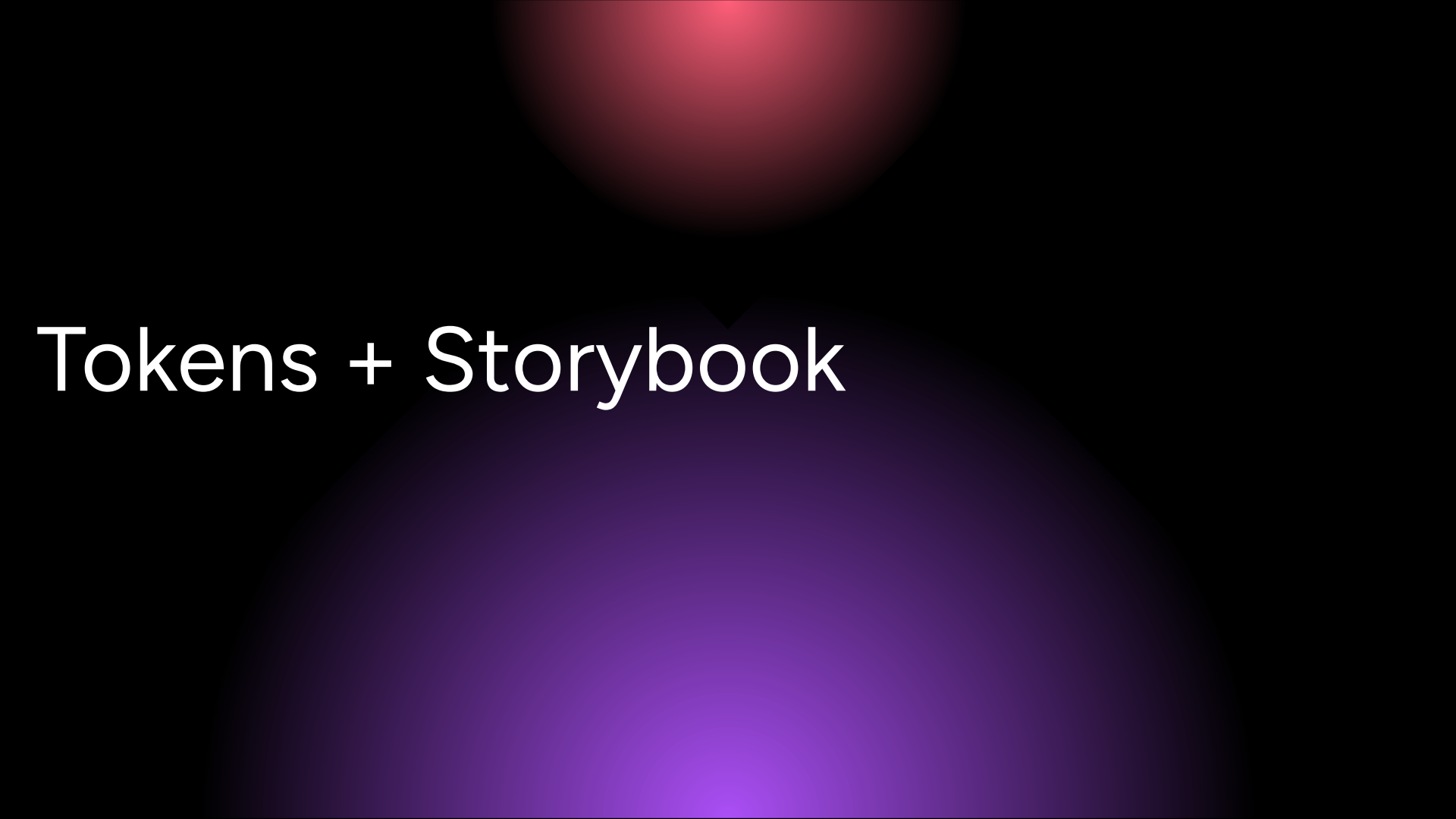
Colors

Spacing

Layout

Rendering

Connecting Design Tokens

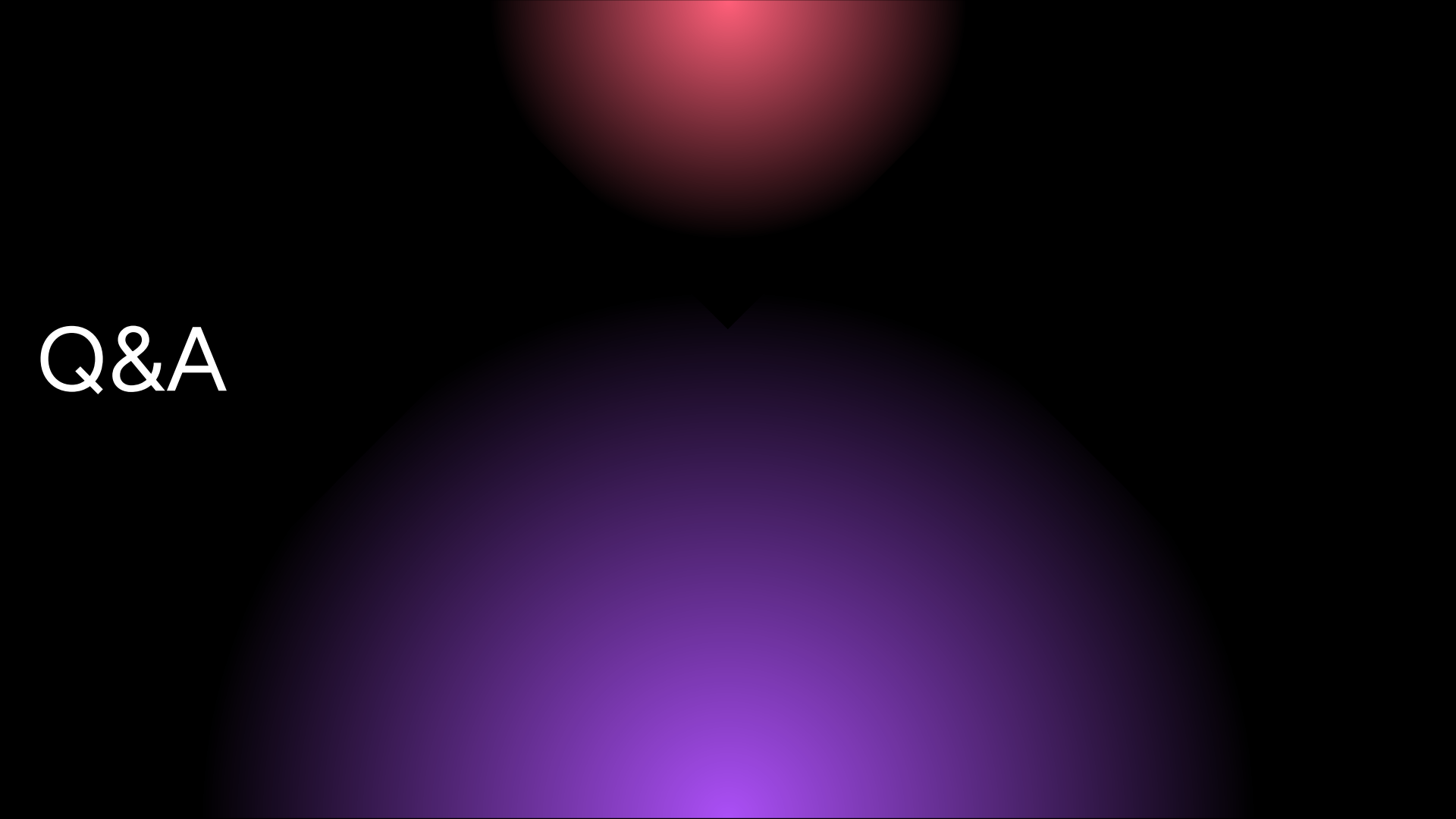


Tokens + Storybook

Getting Adoption



Storybook is Useful



Q&A